ilLog: Incremental Learning based Anomaly Detection from Evolving System Logs

Jiyu Tian, Mingchu Li, Zumin Wang, Liming Chen, *Senior Member, IEEE*, Jing Qin, Jianyuan Gan

***Abstract*—Log anomaly detection (LAD) is of paramount importance to enhance the reliability and stability of software systems. Current state-of-the-art LAD suffers a significant performance degradation when dealing with consistently evolving log events caused by system updates. To build a reliable LAD model under the context of log data evolution, we propose an incremental learning-based method for LAD, namely ilLog, to avoid catastrophic forgetting of previously learned knowledge while continuously updating the model for better detection when processing the evolving log events. In particular, we design a novel entropy-driven sorting algorithm for real log sample replay, which enables the preservation of old knowledge via storing representative samples with discrete sequence features from previous tasks. Additionally, we introduce a Halton-based low discrepancy sequence to better approximate the sliced Cramér distance between the probability distributions of two models, thus enhancing the model learning capability. Based on a standard incremental learning protocol setting, we evaluate the newly proposed ilLog method on three publicly available datasets. Experimental results demonstrate that our approach achieves the best performance compared to SOTA LAD methods and models by applying existing IL-based methods in evolving software systems.**

***Index Terms*—Log Anomaly Detection, Incremental Learning, Information Entropy, Quasi-Monte Carlo.**

I. Introduction

LOG anomaly detection (LAD), aiming to identify abnormal behaviors from log files, has been an active research topic to enhance the security and reliability of software systems [1]–[5]. Recently, deep learning based methods, e.g., DeepLog [6], LogAnomaly [7], LogRobust [8], PLELog [9], MetaLog [10], LogSer [11] and CSCLog [12], have achieved impressive performance to detect log anomalies in complex software systems. They learn effective feature representations from log events and classify the log data into some predefined anomaly categories.

However, these LAD models suffer significant performance degradation when the systems evolve as modern software systems are updated quickly and consistently. System update is a common practice since developers introduce new features and functions, fix bugs and enhance the security of software systems regularly. Consequently, log events for LAD are changed accordingly. Taking Blue-Gene/L. (BGL), a supercomputing operation system as an example, the large number of new log events are consistently defined in the standard log dataset of BGL during its evolution [13]–[16]. Compared to the log event types in the first month, the number of log event types doubled after six months. Figure 1 further shows that the generation and evolution of log events as features of sequential samples may lead to changes in the data distribution. Assume that a LAD method is trained based on log data that contains events at a particular version, its performance will degrade over time when it takes unseen log events in future versions as its inputs.

Incremental learning (IL) is an emerging machine learning paradigm to deal with the issues caused by evolving envi-

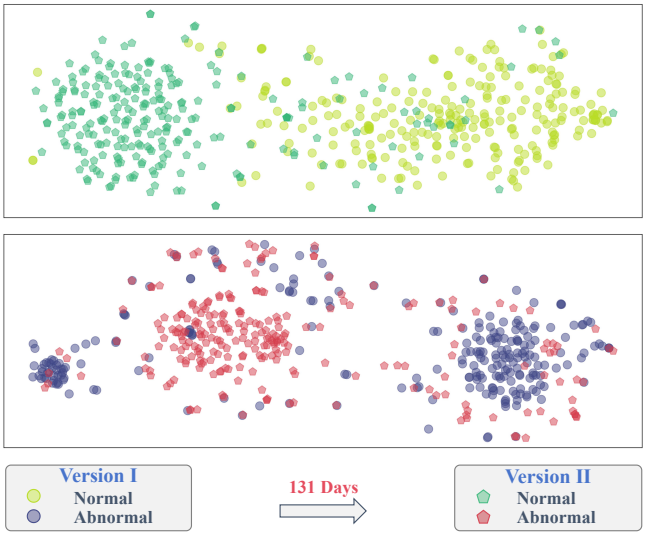


Fig. 1: An example of BGL software system data distribution evolution. Group the parsed BGL dataset based on a sliding window (window size=20, step=10), and construct two dataset versions in chronological order. Each version of the dataset contains 94933 samples, with a 131-day gap between the versions. Randomly sample 500 samples from each version (half normal samples and half abnormal samples), and visualise them using T-SNE. It can be observed that as time progresses, there are significant changes in the data distribution between samples of the same type.

This paper is supported by the National Nature Science Foundation of China under Grant Number: 625B2031 and 62466025. (*Corresponding author: Mingchu Li and Liming Chen*)

Jiyu Tian, Mingchu Li and Jianyuan Gan are with the School of Software Technology, Dalian University of Technology, Dalian, China. Mingchu Li is also with the School of Computer and Information Engineering, Jiangxi Normal University, Nanchang, Jiangxi, China. Email: tianjiyu@mail.dlut.edu.cn

Zumin Wang is with College of Information Engineering, Dalian University, Dalian, China. Email: wangzumin@dlu.edu.cn

Liming Chen is with the School of Computer Science and Technology, Dalian University of Technology, Dalian, China. Email: limingchen0922@dlut.edu.cn

Jing Qin is with School of Software Engineering, Dalian University, Dalian, China. Email: qinjing@dlu.edu.cn

ronmental contexts when processing a large number of new tasks [17]. It continuously learns new knowledge as well as avoids forgetting old knowledge. Thus, the main challenges an IL-based model addresses can be summarised as:

- **Intransigence** refers to the difficulty of incorporating new knowledge into a model that has already been learned from previous data. When confronted with new information, the model exhibits resistance to updating its existing knowledge or adjusting its representations. This can lead to the model being unable to adapt effectively to new tasks, resulting in performance degradation. For LAD, the training cost incurred by the continuous accumulation of training data is unacceptable for the development and maintenance of large-scale software systems [8], [13], [18].
- **Catastrophic forgetting** refers to the severe forgetting or loss of previously learned knowledge by a model when it learns new tasks. When the model is trained on new data, it often overwrites or interferes with the representations and parameters learned in previous tasks. This can cause a sharp decline in performance on previously learned tasks, making it unable to retain its knowledge in continuous learning. For LAD, if access to or utilization of old training data is limited, it inevitably results in forgetting previously learned knowledge [17].

Current state-of-the-art (SOTA) IL-based methods either use memory replay [19]–[21], or update trivial synapse but preserve important parameters of a model via regularization [22]–[26] during the training process. Despite the fact that IL is a suitable framework to tackle the problem of the LAD in an evolving system, it is not straightforward to apply existing IL methods to the task directly due to its unique characteristics. In particular, log samples comprise sequences of different message events, including both discrete features and semantic information. Presently, incremental learning in the context of memory replay focuses mainly on exploring continuous features which makes it difficult to apply existing memory replay methods to log data. Furthermore, parameter regularization-based methods usually require extensive sampling (e.g. Monte Carlo) to approximate the distance between probability distributions as a part of loss. However, the continuous update process of the model is constrained by computational resources due to the massive scale of log data. Random sequences contain clustering and biased sampling points, and may not be efficient, leading to significant wastage of computational resources.

To overcome the above-mentioned limitations, in this work, we propose a novel incremental learning-based log anomaly detection method, ilLog, to enhance the performance of a LAD model in an evolving system. Specifically, we first design an Entropy-Sorting Replay (ESR) algorithm to select log samples with discretized sequence features from previous tasks, obtaining valuable instances that are merged into the training set of the current task. Furthermore, we introduce a Quasi-Monte Carlo (QMC) algorithm based on the Halton low-discrepancy sequence to provide a better approximation of the sliced Cramér distance between the probability distributions of two models. This distance is then integrated as a regularization term in the loss function, with the objective of minimizing changes to the parameters in the model that have a significant impact on previous tasks. To evaluate the proposed approach, we conducted experiments on three publicly available log datasets and performed comparisons with existing log anomaly detection methods. The experimental results indicate that our method achieves better detection performance in terms of accuracy and robustness under the context of an evolving system.

The main contributions of our work can be summarised as follows:

- To the best of our knowledge, this is the first endeavor to introduce incremental learning methods into log anomaly detection to overcome the limitations of existing approaches when adapted to evolving software systems.
- We design an entropy-driven sorting algorithm for real log replay, where we perform under-sampling ($\leq 10\%$) on the log samples from the previous task. We select representative log samples and merge them into the current task, thereby reducing storage and computational costs.
- We present a Quasi-Monte Carlo Scliced Cramér Preservation (QMC-SCP) loss function based on the Halton low-discrepancy sequence to improve the computation of high-dimensional Cramér distance. This method allows an effective measurement of the parameter importance of an evolving model while avoiding catastrophic forgetting.
- We evaluate the effectiveness of our method on three publicly available log datasets by comparing to the SOTA log anomaly detection methods, and the IL-based LAD models by applying existing incremental learning methods to the task.

II. Related Work

A. Log-based Anomaly Detection

Deep learning methods have demonstrated strong capabilities in log anomaly detection [13], [27]–[31]. Currently, log anomaly detection is predominantly composed of three main approaches: unsupervised learning [6], [7], [32], [33], supervised learning [8], [34]–[36], and semi-supervised learning methods [9], [14], [18].

For unsupervised LAD methods, DeepLog [6] employed One-Hot encoding to vectorize log events, transforming log sequences into sparse matrices that do not contain semantic information. LogAnomaly [7] recognized the semantic similarity between log events and utilized the word2vec algorithm to map log events into semantic vectors, enriching the feature representation by incorporating quantitative vectors. DeepSyslog [32] adopted character-level word embedding techniques to uncover log changes through the evolution of log print statements. It extracts semantic and contextual information that is concealed within the log stream to represent the original logs. Besides, Log2Vec used an OOV word processor to embed OOV words at runtime, effectively handling continuously evolving log types.

For semi-supervised LAD methods, SwissLog [18] combined semantic embedding and temporal embedding to train a unified attention-based Bi-LSTM model for capturing anomalies. PLELog [9] incorporated historical anomaly knowledge through probability label estimation and utilizes semantic embedding to mitigate the impact of unstable log data.

For supervised LAD methods, LogRobust [8] also addressed the issue of log evolution. This method combined TF-IDF and FastText techniques to mitigate the effects of log evolution and parsing noise, and introduces an attention mechanism within the Bi-LSTM architecture to train a supervised detection model. LightLog [34] extended the methods from LogRobust for handling evolving logs and introduced semantic vector dimensionality reduction and a lightweight TCN network, significantly reducing the floating-point computations and detection time. NeuralLog [35] required no prior parsing and could directly extract semantic vectors from raw logs. It then used a transformer-based classification model to detect anomalies.

Recently, Le et al. [27] pointed out that existing methods still face challenges regarding limited labeled data, early detection, system evolution, and the relations among log events. Of these, the dynamic evolution of the system has garnered significant attention. Nonetheless, a static training framework fails to learn anomalous patterns evolving from new events, which may result in a decrease in the generalization ability and detection performance of the old model on new instances.

B. Incremental learning

Incremental learning is capable of processing continuous streams of information, retaining and integrating prior knowledge while absorbing new knowledge [37]. Based on mechanisms for preventing forgetting, incremental learning methods can be divided into three categories: Parameter regularization-based methods [22]–[26], [38], replay-based methods [19]–[21], [39], and parameter isolation-based methods [40]–[43].

Parameter regularization-based incremental learning primarily protected existing knowledge by imposing constraints on the loss function of the new task. This approach can be further divided into two types: Prior-focused and Data-focused. Prior-focused approaches aim to penalize changes in important parameters to prevent significant alterations in parameters related to previous tasks. EWC [22] was the first to introduce the Prior-focused concept and used the second-order derivative of the loss with respect to the parameters to measure their importance. MAS [23] recorded the sensitivity of the prediction output to parameter changes and accumulated an importance measure for each parameter in the network. SCP [25] demonstrated the close relationship between methods like EWC and MAS and further consolidated model parameters by using the Cramér distance to measure the difference between two probability distributions. Data-focused approaches are based on the idea of knowledge distillation to preserve the knowledge learned from previous tasks. LwF [38] minimized the Kullback-Leibler (KL) divergence between the probability distributions of the old and new model outputs. Foster [26] dynamically expanded new modules to fit the residuals between the target and the output of the original model and employed an effective distillation strategy to maintain a single backbone model.

Replay-based incremental learning trains new tasks while retaining a portion of old representative data for the model to review previously learned knowledge. iCaRL [19] introduced the concept of replay-based incremental learning. After completing each task, this method saved a few samples from each class for subsequent model training. Considering the class imbalance problem due to storage limitations, Rwalk [20] stored samples with high entropy of softmax output probabilities or samples that are close to the decision boundary for replay. BiC [39] explicitly calibrated the model's output by learning a linear bias correction layer using an additional balanced validation set. WA [21] reduced the imbalance between old and new classes by aligning the logits outputted by the model on old and new class nodes.

Parameter isolation-based incremental learning is to incrementally expand the model while keeping the parameters of the old task unchanged. This involves isolating the parameters of the old and new tasks. DER [41] froze the previously learned feature extractor and expanded the super feature extractor network with a new feature extractor when facing a new task. Finally, the features extracted by all the extractors are concatenated together for classification. DCE [42] addressed class imbalance in domain-incremental learning by employing a two-stage process: training a set of expert networks respectively biased towards the majority, balanced, and minority classes, and adapting to distribution shifts with a dynamic selector. SOYO [43] balanced data by compressing historical features using a Gaussian Mixture Model for resampling, then extracted discriminative features through a multi-level fusion network.

Logs are discrete feature samples with semantic similarities, which are rarely considered by current memory replay studies. Additionally, the computational complexity of the loss function in parameter regularization methods makes it challenging to continuously train on large-scale data. Therefore, further exploration is needed for the application of existing incremental learning methods in the field of log anomaly detection.

III. Methodology

A. Problem Formulation

The incremental learning task, when referring to log anomaly detection, can be defined as the ability of a classification model to learn the new anomaly patterns without forgetting or deteriorating too much the performance of previously learned ones. Consequently, the main objective then is to strike a balance between the preservation of previous knowledge and the capability of learning new events and patterns.

In the incremental log anomaly detection scenario, we define log anomaly detection as a task flow, where each task corresponds to a group with the same label. For the $\mathcal{K}$-th task, we define $\mathcal{D}^{\mathcal{K}} = \{(\mathcal{X}_i^{\mathcal{K}}, \mathcal{Y}_i^{\mathcal{K}})\}_{i=1}^{\mathcal{N}}$ as the training set. where $\mathcal{X}_i^{\mathcal{K}} \in \mathcal{X}$ represents the input and $\mathcal{Y}_i^{\mathcal{K}} \in \{0, 1\}$ represents the true label, indicating that the output space consists of two labels: normal and abnormal. We assume that the sampler

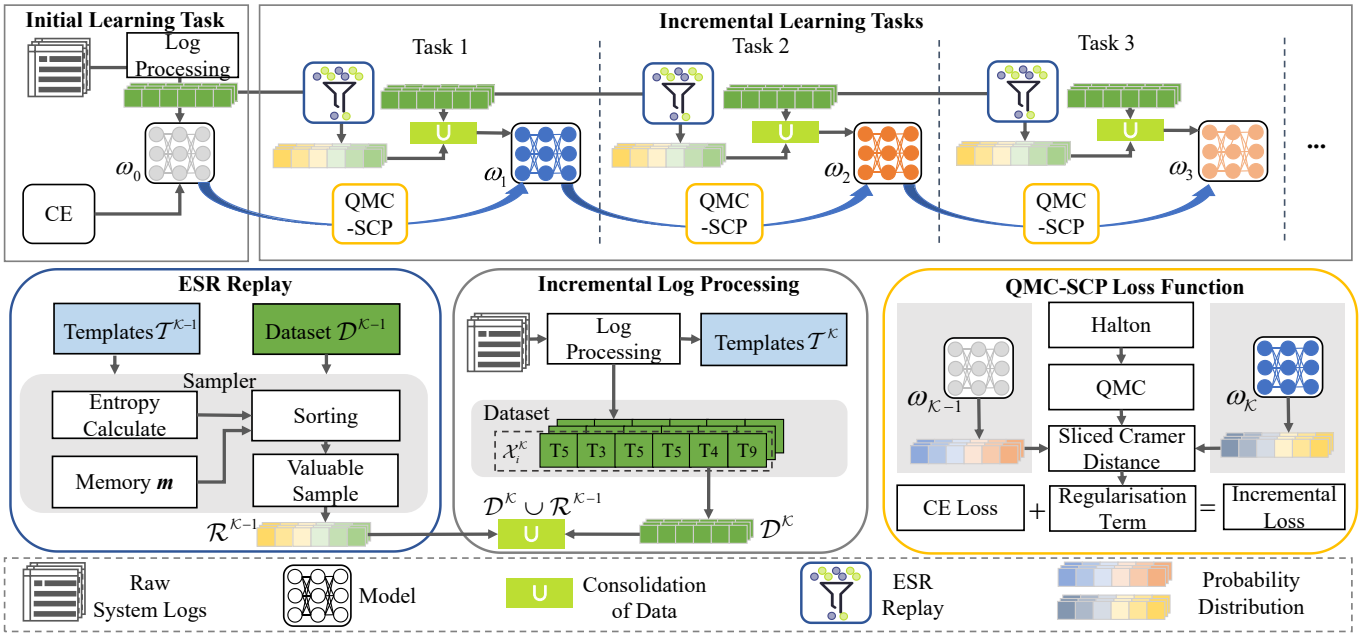


Fig. 2: Overview of ilLog approach. The learning model of ilLog is categorized into two task types: initial learning task and incremental learning task, where incremental learning task consists of three essential components: incremental log processing, ESR, and QMC-SCP loss function.

$\mathcal{R}^{\mathcal{K}} = \{(\mathcal{X}_i^{1:\mathcal{K}-1}, \mathcal{Y}_i^{1:\mathcal{K}-1})\}_{i=1}^{\mathcal{N}}$ contains the replay data of previous tasks, where $1{:}\mathcal{K}$ represents from the first task to the $\mathcal{K}$-th task. The optimization problem is defined as follows:

$$\arg\min_\theta \mathbb{E}_{(\mathcal{X},\mathcal{Y})\sim\mathcal{D}^{\mathcal{K}}\cup\mathcal{R}^{\mathcal{K}}}\left[\mathcal{L}\left(\mathcal{X},\mathcal{Y};\theta\right)\right] \tag{1}$$

where the $\mathcal{L}$ is the loss function, typically includes a base loss (e.g., Cross-Entropy loss) and a regularization term, and θ is the parameters of the model.

B. Incremental Log Anomaly Detection Framework

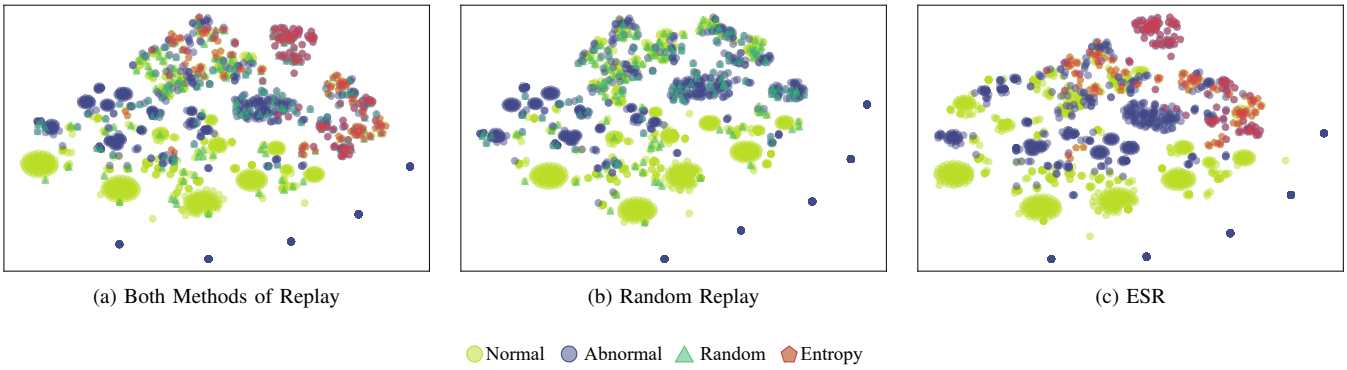
Our incremental learning log anomaly detection framework, namely ilLog, is shown in Figure 2, where iLog is a continuous log anomaly detection framework that divides the training and detection process into multiple tasks based on chronological. According to the training mode, tasks can be classified into two categories: initial learning tasks and incremental learning tasks. In the initial learning task, since there is no prior data, the detection model only needs to be trained following the common training mode without considering catastrophic forgetting. In the incremental learning tasks, we provide a data replay strategy based on entropy sorting and a parameter regularization loss function called QMC-SCP to combat catastrophic forgetting, ensuring the continuous generalization ability of the detection model towards evolving logs.

Initial Learning Task The initial learning task only occurs during the first training phase. Given the absence of prior data, it is not necessary to implement any measures to prevent knowledge forgetting during this particular stage. The raw log data in the system undergoes several steps of log processing, including parsing, grouping, and representation (Since it is not the focus of this paper, we followed the details of [27] for log processing). These steps transform the raw log data into log sequence samples, which serve as the training set for the model. The detection model is trained using the cross-entropy loss function without regularization term.

Incremental Learning Task Each incremental training task consists of three components: incremental log processing, ESR algorithm, and QMC-SCP loss function.

Component 1: Incremental Log processing involves preprocessing the log messages from the new task. The raw logs generated by the software system undergo parsing, grouping, and representation processes to transform them into a template set and dataset. As shown in Figure 2, ilLog then merges the dataset $\mathcal{D}^{\mathcal{K}}$ with the valuable training samples $\mathcal{R}^{\mathcal{K}}$ that have been accumulated from previous tasks (i.e. $\mathcal{D}^{\mathcal{K}} \cup \mathcal{R}^{\mathcal{K}-1}$). This merged dataset will serve as the training set for the current task, allowing for a comprehensive and robust learning process. Moreover, both the dataset $\mathcal{D}^{\mathcal{K}}$ and the template set $\mathcal{T}^{\mathcal{K}}$ from the current phase will be utilized for ESR algorithm in the subsequent tasks, ensuring the retention and transfer of knowledge acquired in this task to future detection task.

Component 2: ESR consists of calculating the importance of the training samples from the previous task, selecting a few valuable samples for storage, and merging them with the training samples from the current task. Creating a training set in this way preserves some prior knowledge, which can be used to mitigate forgetting and significantly reduce computational costs. ESR algorithm leverages entropy as a metric to assess the value of log sequence samples. Higher entropy indicates greater training value, meaning that samples with higher entropy are considered more valuable for learning. Additionally, the number of samples retained in the replay process is determined by the memory m, which serves as a



(a) Both Methods of Replay (b) Random Replay (c) ESR

Fig. 3: Intuitive comparison of feature distributions among different data replay algorithms based on T-SNE.

criterion for selecting the optimal number of samples to retain. Notably, unlike randomly selected samples, the algorithm focuses on selecting distinctive samples (details are presented in Section IV).

Component 3: QMC-SCP loss function with parameter regularization is crucial for preserving important model parameters to mitigate catastrophic forgetting. Unlike typical loss functions (e.g., cross-entropy), the incremental loss function suppresses changes in parameters that have a significant impact on previous tasks by adding a regularization term. It only adjusts the less important parameters for the previous detection task in the current training phase. Our proposed QMC-SCP achieves a more accurate calculation of the Cramér distance between probability distributions from two different models ($\omega_{\mathcal{K}}$ and $\omega_{\mathcal{K}-1}$) by introducing a Halton-based low variance sequence instead of a Monte Carlo random sequence (details are presented in Section V).

IV. Log Samples Replay based on Entropy Sorting

Although memory replay techniques have been explored in incremental learning tasks, there has been relatively less research on applying these techniques to discrete features.

A. Intuitive Explanation

Preserving old data is essential for maintaining previous knowledge. One effective method is to randomly select samples from all training samples of each class. This random selection process helps to represent the task mean, ensuring that important characteristics and patterns from the old data are retained. However, we rely on two intuitions: Firstly, scarce samples may contain rare but significant exceptional information. Smooth-running software systems produce a fixed type of log event over time. The generation of a rare event often means that an unusual change has occurred in the system and the information it represents is more meaningful. Secondly, due to the changes in model parameters being regularized (see Section IV), the feature space and decision boundaries do not change significantly. Therefore, the replay strategy should select samples near the classification decision boundaries [20].

We assume that samples near the decision boundaries are more representative than samples far from the decision boundaries. As shown in Figure 3, compared to random selection, the visualization results demonstrate that the sample distribution selected using the entropy-based replay sampling strategy is located near the boundaries of the classes.

B. Theoretical Analysis of Entropy Sorting Replay

Based on the definition of information entropy, here are some conclusions. For a log event, if it appears less frequently in the log sequence, its probability of occurrence is lower, resulting in higher self-information. As a result, the average information (i.e., entropy) of the log sequence containing that event is larger. On the other hand, if the event appears more frequently in the log sequence, its probability is higher, leading to lower self-information. Consequently, the information entropy of the log sequence containing that event is smaller. Therefore, we determine the importance of a log sequence by calculating its entropy in the training set.

C. Entropy Sorting Replay Algorithm

Here we provide a specific explanation of how to calculate the importance of samples composed of different log events. Assuming in task $\mathcal{K}$, we have a raw log set from the previous task, which contains d log messages and various types of templates (i.e., log events). The template set containing all event types from the current task and previous tasks is denoted as $\mathcal{T}^{\mathcal{K}} = \{\mathcal{T}_i\}_1^{\mathcal{I}}$. For a simpler description, we implement grouping with a fixed window and set the window size to be $\mathcal{W}$. Then, we can obtain a dataset $\mathcal{D}^{\mathcal{K}-1} = \{\mathcal{S}_n\}_1^N$ that contains $\mathcal{N}$ log sequence samples, where $\mathcal{N} = \lfloor d/\mathcal{W} \rfloor$. For a log sequence, since the window size is fixed as $\mathcal{W}$, thus $\mathcal{S}_n = \overrightarrow{[\mathcal{E}_1, \mathcal{E}_2, \mathcal{E}_3, ..., \mathcal{E}_w]}$, $\mathcal{E}_w \in \mathcal{T}^{\mathcal{K}}$, $w \in [\mathcal{W}]$ which means a log sequence contains $\mathcal{W}$ log events. The probability of each log event $\mathcal{E}_w$ occurring is given by:

$$\mathcal{P}(\mathcal{E}_w) = \frac{\sum_{n=1}^{N} \sigma_{n,w}}{\mathcal{N}} \tag{2}$$

when $\mathcal{E}_w \in \mathcal{S}_n$, $\sigma_{n,w} = 1$, , otherwise $\sigma_{n,w} = 0$. Thus, the formula for the self-information of event $\mathcal{E}_w$ is as follows:

$$\begin{aligned}\mathcal{I}(\mathcal{E}_w) &= -\log \mathcal{P}(\mathcal{E}_w) \\ &= -\log\left(\frac{\sum_{n=1}^{\mathcal{N}} \sigma_{n,w}}{\mathcal{N}}\right) \\ &= \log\left(\frac{\mathcal{N}}{\sum_{n=1}^{\mathcal{N}} \sigma_{n,w}}\right)\end{aligned} \tag{3}$$

Entropy is used to measure the average information of a sample, which represents the average expectation of the information. For each log sequence $\mathcal{S}_n$, the formula for calculating its entropy is:

$$\begin{aligned}\mathcal{H}(S_n) &= -\sum_{w=1}^{\mathcal{W}} \mathcal{P}(\mathcal{E}_w)\mathcal{I}(\mathcal{E}_w) \\ &= -\sum_{w=1}^{\mathcal{W}} \mathcal{P}(\mathcal{E}_w)\log \mathcal{P}(\mathcal{E}_w) \\ &= \sum_{w=1}^{\mathcal{W}} \frac{\sum_{n=1}^{\mathcal{N}} \sigma_{n,w}}{\mathcal{N}} \log\left(\frac{\mathcal{N}}{\sum_{n=1}^{\mathcal{N}} \sigma_{n,w}}\right)\end{aligned} \tag{4}$$

where $\mathcal{N}$ is the number of log sequences included in the dataset $\mathcal{D}^{\mathcal{K}-1}$, which can be omitted in the probability operations, thus

$$\mathcal{H}(\mathcal{S}_n) \propto \sum_{w=1}^{\mathcal{W}} \left(\sum_{n=1}^{\mathcal{N}} \sigma_{n,w} \log\left(\frac{1}{\sum_{n=1}^{\mathcal{N}} \sigma_{n,w}}\right)\right) \tag{5}$$

Then, we obtain the information entropy set $\mathcal{H}^{\mathcal{K}-1} = \{\mathcal{H}(\mathcal{S}_1),\ \mathcal{H}(\mathcal{S}_2),\ ...,\ \mathcal{H}(\mathcal{S}_n)\}$ for the log sequence samples. Therefore, the order of the sample sequences constructed based on entropy values is meaningful, where the samples with higher entropy can be considered more important and contribute more to the training. Memory *m* denotes the maximum number of samples that can be retained for each class in the previous task, which is usually less than 10% of the total number of samples of that class. Therefore, we sequentially select *m* samples with the highest entropy value from $\mathcal{H}^{\mathcal{K}-1}$ as the replay dataset $\mathcal{H}_m^{\mathcal{K}-1}$ for the previous task. Therefore, the final output of the data replay is

$$\mathcal{R}^{\mathcal{K}} = \mathcal{H}_m^{\mathcal{K}-1} \cup \mathcal{H}_m^{\mathcal{K}-2} \cup \mathcal{H}_m^{\mathcal{K}-3} \cup \cdots \cup \mathcal{H}_m^{1} \tag{6}$$

Thus, the merged training set $\mathcal{D}_{\mathcal{M}}^{\mathcal{K}}$ of current task can be formula as

$$\mathcal{D}_{\mathcal{M}}^{\mathcal{K}} = \mathcal{D}^{\mathcal{K}} \cup \mathcal{R}^{\mathcal{K}} \tag{7}$$

V. QMC-SCP Loss function for Balancing Intransigence and Forgetting

A. SCP Loss Function

As described in Section II-B, the parameter isolation-based approach has limitations when it comes to handling a large number of successive tasks, and the replay-based approach is utilized and elaborated in Section IV. Therefore, during incremental training, we employ a regularization-based incremental loss function to address the forgetting-intransigence dilemma. In comparison to the commonly used KL-divergence [22], [23] and its symmetric form, Jensen-Shannon distance, Cramér distance respects the underlying geometry of the problem, offers improved performance in capturing the differences between distributions [25].

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathcal{B}}(\theta) + \lambda \mathcal{L}_{\mathcal{B}\to\mathcal{A}}(\theta) \tag{8}$$

As shown in Equation (8), the total loss function $\mathcal{L}(\theta)$ comprises two components: the cross-entropy loss term $\mathcal{L}_{\mathcal{B}}(\theta)$ and the regularization term $\mathcal{L}_{\mathcal{B}\to\mathcal{A}}(\theta)$. where the hyperparameter λ is used to balance these two components. If we set λ = 0, it indicates that we do not apply parameter-regularization (10) and only use cross-entropy loss (9) for model training, which can lead to catastrophic forgetting in incremental learning tasks.

$$\mathcal{L}_{\mathcal{B}}(\theta) = -\frac{1}{\mathcal{N}} \sum_{i=1}^{\mathcal{N}} \mathcal{Y}_i log(\hat{\mathcal{Y}}_i) \tag{9}$$

$$\mathcal{L}_{\mathcal{B}\to\mathcal{A}}(\theta) = \mathcal{SC}(p^{\mathcal{A}}(\cdot|\theta_{\mathcal{A}}^*), p^{\mathcal{A}}(\cdot|\theta)) \tag{10}$$

To overcome forgetting, SCP [25] leverages the radon transform to slice the Cramér distance (SC) between high-dimensional distributions, aiming to make the new probability distribution $p^{\mathcal{A}}(\cdot|\theta_{\mathcal{A}}^*)$ close to the $p^{\mathcal{A}}(\cdot|\theta)$ learned from previous tasks $\mathcal{A}$. As learning progresses, this approach preserves the inherent properties of the model regarding the previous tasks. After further derivation based on Equation (10) (see [25]), the regularized term is obtained as follows:

$$\mathcal{L}_{\mathcal{B}\to\mathcal{A}}(\theta) = \frac{1}{\mathcal{Q}} \sum_{q=1}^{\mathcal{Q}} \left(\frac{1}{\mathcal{N}} \sum_{n=1}^{\mathcal{N}} \frac{d\xi_q \cdot \phi(\mathbf{x}_n^A; \theta_A^*)}{d\theta_i}\right)^2 \tag{11}$$

The SC distance between conditional probability distributions is difficult to compute directly. Therefore, SCP uses the Monte Carlo algorithm to obtain approximate solutions for the high-dimensional integral of the SC. $\phi(\mathbf{x}_n^A; \theta_A^*)$ is the output of the network. ξ_q is to slice the average output using a Monte Carlo algorithm based on pseudo-random sequences.

B. Halton-Based Quasi-Monte Carlo

However, when it comes to Monte Carlo methods, the utilization of pseudo-random sequence sampling in computer-based random sampling can result in clustering and bias within the sampled points. Firstly, the bias of sampling points directly impacts the accuracy of the approximate solution for sliced Cramér distance. Secondly, the clustering of sampling points generates redundant data, leading to significant computational resource wastage. When handling a vast amount of log samples, such inefficiency is deemed highly unacceptable.

Therefore, we introduce Quasi-Monte Carlo (QMC) with Halton-based low-discrepancy sequences as the sampling method to approximate the solution of higher dimensional integrals. Compared to the randomly generated sequences, the sampling points of the Halton sequence exhibit advantages

TABLE I: Description of the log dataset used in the experiments

Datasets	Parsing	Events	Grouping	Window Size	Step	Types	Samples
HF	Public [27]	28	Session	/	/	Normal Abnormal	575,059 16,838(2.8%)
BGL	Public [27]	378	Sliding	20	10	Normal Abnormal	433,877 40,789 (8.6%)
TB	Spell [44]	6,411	Sliding	20	10	Normal Abnormal	749,681 247,831 (24.8%)

such as uniform distribution, low repetitiveness, and low density. In QMC, any non-negative integer a can be represented as a string of digits in base b. For example, *6* can be represented as *110* in base *2*. The radical inverse function Φ_b represents the reversal of a number string in base b, and then converts the resulting number string into a fraction by adding a 0. The result falls within the interval [0, 1), and it can be expressed mathematically as:

$$\Phi_b(a) = 0.b_1(a)b_2(a)...b_n(a) = \sum_{i=0}^{i} b_i(a)b^{-i-1} \tag{12}$$

where the Halton sequence is implemented based on a specific set of prime numbers, with each dimension corresponds to a prime number.Typically, the values are taken in the order of the prime numbers. This approach facilitates the control of distances between different sampling points when combined across dimensions.

C. QMC-SCP for overcoming forgetting

Combined with the Quasi-Monte Carlo solution, we end up with the following improved loss function regularization term $\mathcal{L}_{B\to A}(\theta)$:

$$\frac{1}{Q}\sum_{q=1}^{Q}\left(\frac{d\left(\sum\limits_{i=0}^{i} b_i(Q)b^{(-i-1)}\cdot\frac{1}{\mathcal{N}}\sum\limits_{n=1}^{\mathcal{N}}\phi\left(\mathbf{x}_n^{\mathcal{A}};\theta_{\mathcal{A}}^{*}\right)\right)}{d\theta_i}\right)^2 \tag{13}$$

The main difference between regularization terms (10) and (13) lies in their respective sampling approaches for approximating high-dimensional integrals. In (10), calculation using ξ_q is based on computer-generated random numbers, while in (13), the calculation is based on low-discrepancy Halton sequences, which allows for obtaining accurate computational results. Where $\Phi_b(q) = \sum_0^i b_i(q)b^{-i-1}$, $\vec{\phi}_{\mathcal{A}}^{*} = \frac{1}{\mathcal{N}}\sum_{n=1}^{\mathcal{N}}\phi\left(\mathbf{x}_n^{\mathcal{A}};\theta_{\mathcal{A}}^{*}\right)$, the obtained total loss function is

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathcal{B}}(\theta) + \frac{1}{\mathcal{Q}}\sum_{q=1}^{\mathcal{Q}}\left(\frac{d\Phi_b\left(q\right)\cdot\vec{\phi}_{\mathcal{A}}^{*}}{d\theta_i}\right)^2 \tag{14}$$

VI. Experiment

In our research, we evaluate our method by addressing the following research questions:

RQ1: How effective is the ilLog method in evolving software systems?

RQ2: How does the ilLog method perform in terms of robustness?

RQ3: How does the ilLog method perform in terms of advancement?

RQ4: How does the ilLog method perform on different backbones?

RQ5: How does the ilLog method perform in terms of incremental training efficiency?

RQ6: How does each component contribute to ilLog?

A. Experimental Setup

1) Dataset: To evaluate the proposed ilLog, we utilize the original HDFS dataset, BGL dataset, and Thunderbird dataset as data sources, and restructured each dataset according to the incremental learning protocol. The detailed information for each dataset is as follows:

HDFS dataset (HF) [45] consists of 11,175,629 logs produced from more than 200 Amazon EC2 nodes, spanning 38.7 hours. These log messages form different log sequences according to their session ID, including 575,059 normal log sequences and 16,838 (2.8%) abnormal log sequences.

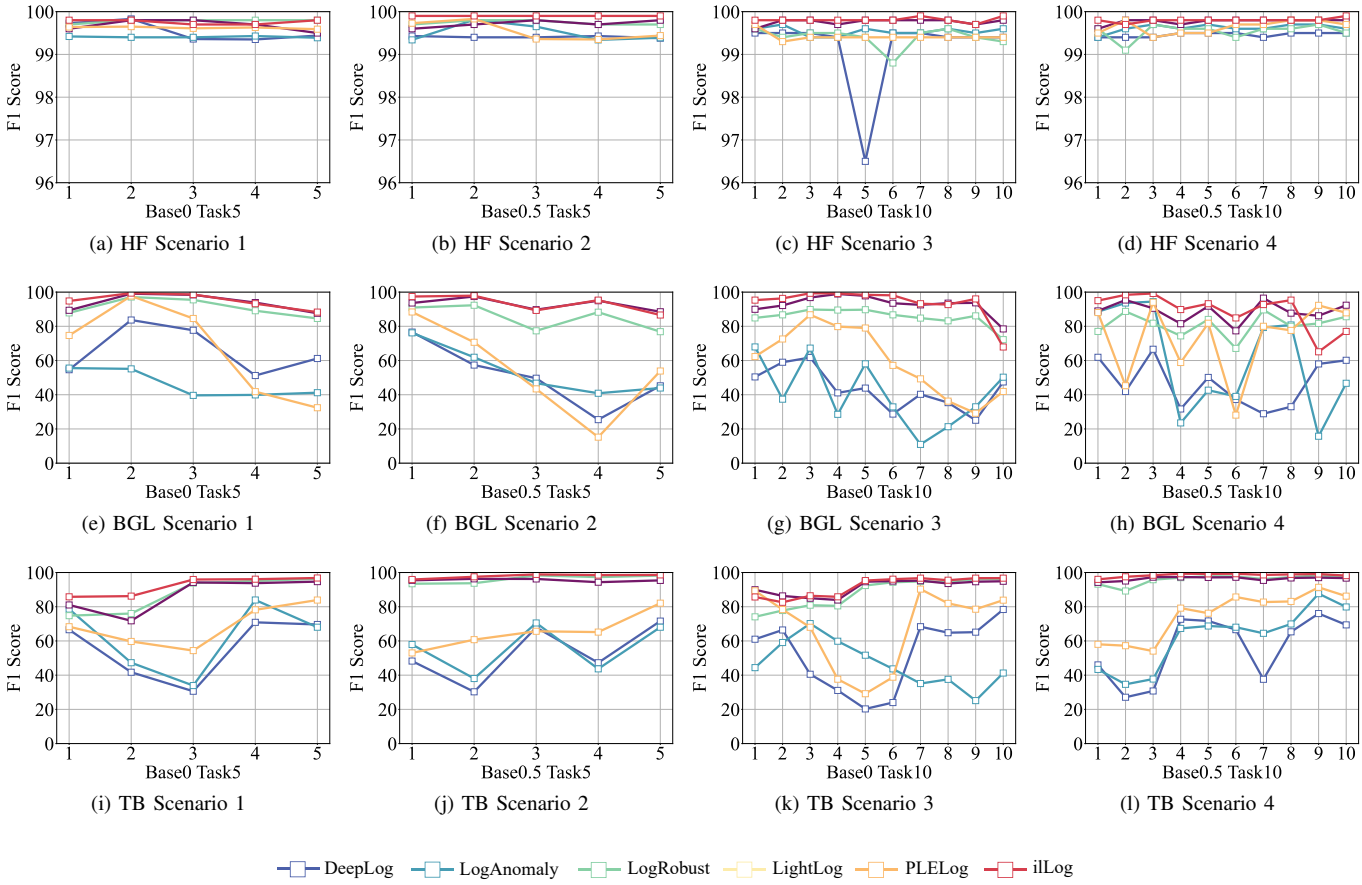
BGL Dataset [46] consists of 4,747,963 log messages generated by a supercomputing system Blue-Gene/L, spanning 244 days. Each message in the BGL dataset was manually labeled as either normal or anomalous. As described in Table I, we set the size of the sliding window to 20 and the step size to 10, a total of 40,789 (8.6%) abnormal sequences were obtained by this method.

Thunderbird dataset (TB) [46] consists of more than 200 million logs collected from a Thunderbird supercomputer at Sandia National Labs (SNL), spanning 38.7 hours. The log data contains normal and abnormal log messages which are manually identified. As the raw dataset is so large that it was difficult to process, we initially selected 10 million consecutive log messages for validation. We set the window size to 20 and the step size to 10, used the sliding window to divide the logs and obtained 247,831 (24.8%) anomalous sequences.

Following the standard setup protocol of incremental learning, we partition the incremental learning task chronologically, dividing each dataset into four testing scenarios: Base0 Task5, which evenly divides each dataset into five consecutive task subsets (T1, T2, T3, T4, T5) based on chronology; Base0.5 Task5, which divides each dataset into five consecutive task subsets while allocating 50% of the samples to the first task subset (T1) and evenly distributing the remaining 50% of samples into four consecutive subsets (T2, T3, T4, T5) based on chronology; Base0 Task10, which evenly divides each

TABLE II: Comparison F1 score with existing LAD approaches

Methods	B0T5			B0.5T5			B0T10			B0.5T10		
	HF	BGL	TB	HF	BGL	TB	HF	BGL	TB	HF	BGL	TB
DeepLog	99.5	65.6	65.6	99.4	50.8	53.0	99.1	43.3	51.9	99.4	46.9	56.3
LogAnomaly	99.4	46.2	46.2	99.5	53.9	53.6	99.5	40.7	46.7	99.6	60.	60.1
LogRobust	99.8	90.8	90.8	99.7	85.1	96.2	99.4	85.4	88.2	99.5	81.0	96.0
LightLog	99.6	93.6	93.6	99.7	92.8	95.5	99.7	92.7	91.3	99.7	88.8	96.4
PLELog	99.6	66.2	66.2	99.5	54.3	57.3	99.4	59.4	67.5	99.6	73.3	66.9
ilLog	**99.5**	**94.8**	**94.8**	**99.9**	**93.3**	**97.9**	**99.8**	**93.6**	**91.8**	**99.8**	**89.1**	**98.4**



(a) HF Scenario 1 (b) HF Scenario 2 (c) HF Scenario 3 (d) HF Scenario 4

(e) BGL Scenario 1 (f) BGL Scenario 2 (g) BGL Scenario 3 (h) BGL Scenario 4

(i) TB Scenario 1 (j) TB Scenario 2 (k) TB Scenario 3 (l) TB Scenario 4

Fig. 4: Comparison of different log anomaly detection methods in evolving scenario

dataset into ten consecutive subsets (T1, T2, T3, T4, T5, T6, T7, T8, T9, T10) based on chronology; Base0.5 Task10, which divides each dataset into ten consecutive task subsets while allocating 50% of the samples to the first task subset (T1) and evenly distributing the remaining 50% of samples into nine consecutive subsets (T2, T3, T4, T5, T6, T7, T8, T9, T10) based on chronology. Among them, each subset contains both a training set and a test set. Here, we set the split value to 0.7, meaning that the training set accounts for 70% of the samples, while the test set accounts for 30% of the samples in each subset. These different testing scenarios correspond to learning to distinguish log events that evolve and incrementally change over time.

2) Evaluation Metrics: Anomaly detection in log data using incremental learning involves the binary classification of sequences, with test results categorized into four outcomes:

- TP: Correctly detecting normal samples as normal
- FN: Incorrectly detecting normal samples as abnormal
- TN: Correctly detecting abnormal samples as abnormal
- FP: Incorrectly detecting abnormal samples as abnormal

To assess the effectiveness of the anomaly detection model in incremental scenarios, the F1 Score is used as a detection metric. This metric is a comprehensive evaluation metric that includes Precision and Recall. Among them, precision is the proportion of all log sequences identified as anomalous that are correctly identified as anomalous. Precision = $\frac{TP}{TP+FP}$. Recall is the proportion of all log sequences that are correctly identified as anomalous. Recall = $\frac{TP}{TP+FN}$. F1 Score (F1) is the summed average of precision and Recall. F1 = $\frac{2*Precision*Recall}{Precision+Recall}$.

3) Implementations and Environments: In our experiments, we process the log data and conduct log-based anomaly

detection as follows: we adopt the implementations of log processing provided by open source [27]. We use Adam as the optimizer, set the learning rate to 0.0001, the number of epochs to 100, the Memory for each task is 10% of the training set, and the sampling value Q to 200. To complete the testing of the method, the experiments are performed using the following hardware and software platforms: Intel(R) Xeon(R) Gold 5218 CPU @ 2.30GHz, CPU cores 16, 256 GB RAM, NVIDIA Tesla V100-PCIE-32GB*2, CentOS Linux release 7.9, CUDA 11.7, cuDNN 7.6.5, Python 3.7.12, and PyTorch 1.7.0.

B. RQ1: Comparison with Different SOTA Log Anomaly Detection Approaches

To evaluate the incremental detection performance of ilLog more intuitively, the following experiments compared it with five state-of-the-art log anomaly detection approaches: DeepLog and LogAnomaly based on unsupervised learning, LogRobust and LightLog based on supervised learning, and PLELog based on semi-supervised learning. The experimental results are shown in Figure 4 and Table II.

For the HF dataset, in the testing scenario of Base0 Task10 (Figure 4 (c)), the F1 score of the DeepLog method is 96.5% in the 5th task, significantly lower than the other methods. In the other three testing scenarios (Figure 4 (a) (b) (d)), all detection methods achieve an F1 above 99%. Therefore, compared to the other five LAD methods, our proposed ilLog method does not show significant advantages in terms of detection performance. However, in consecutive detection tasks, our method demonstrates the most stable performance. prior work [14], [16], we analyze that this is because the log events in the HF dataset have minimal dynamic evolution. This indirectly verifies that existing log anomaly detection methods can perform well in detecting performance for a non-evolving software system.

For the BGL dataset, as expected, in the evolving logs, our proposed detection method based on parameter regularization and data replay outperforms other detection methods by a large margin (see Figure 4 (e) (f) (g) (h)). Furthermore, the ilLog method, while ensuring generalization to new task log events, does not forget the knowledge learned from previous tasks. As shown in Figure 4 (e), when transitioning from Task 2 to Task 3, ilLog quickly adapts to the new task, achieving an F1 of over 99%; when transitioning from Task 5 to Task 6, the detection performance of the current supervised learning methods rapidly deteriorates, but ilLog's detection performance remains unaffected. Therefore, in most task transitions, our method exhibits a lower forgetting rate of previous knowledge and faster adaptation to new knowledge, resulting in the overall best performance.

For the TB dataset, as shown in Figure 4 (i) (j) (k) (l), ilLog maintains the highest detection accuracy in most tasks. Additionally, it is easy to observe that compared to supervised learning models, unsupervised and semi-supervised learning models are lacking in both detection accuracy and the ability to handle adversarial log evolution. For example, DeepLog and LogAnomaly exhibit a sharp drop in detection accuracy to below 50% for most tasks.

C. RQ2: Robustness Analysis of Different Log Anomaly Detection Approaches

Mislabeled and parsing errors are prevalent forms of noise often encountered in log data [27]. Specifically, mislabeled errors possess the potential to disrupt the classification boundaries established by detection models, while parsing errors have the propensity to introduce spurious log events and generate erroneous log templates. Consequently, to assess the robustness of our approach, we conduct an evaluation encompassing various levels of mislabeled logs alongside diverse log parsers. In the mislabeled experiments, we introduce logs comprising 5% and 10% of the entire dataset, with an equal distribution of normal and abnormal labels, into the training data. Meanwhile, the test set remained unaltered. In the parsing error experiments, we employ two widely utilized log parsing techniques: Drain [47] and Spell [5]. These experiments are conducted on two datasets, namely BGL and TB. As illustrated in Figure 5, the noise-resistant capabilities of our approach are demonstrated to exhibit a notable advantage.

Figures 5 (a)-(d) depict the impact of different percentages of mislabeled logs on the precision, recall and F1 of the model. It is evident that as the percentage of mislabeled logs rises from 5% to 10%, the performance of the models experiences varying degrees of decline. On the BGL dataset, LogRobust and LightLog exhibit greater sensitivity to mislabeled logs. When the mislabeled logs account for 5%, the proposed ilLog method achieves an impressive F1 of 92.1%, surpassing the scores of 89.1% for LogRobust and 90.4% for LightLog. However, when the mislabelling ratio increases to 10%, the F1 of LogRobust and LightLog plummets to 78.1% and 81.7%, respectively. With essentially equal precision ratios, the proposed ilLog maintains a substantial recall of 87.6%, outperforming LogRobust by 22.1% and LightLog by 16.6%. Notably, despite a marginal decrease of only 0.2% in the F1, ilLog showcases remarkable robustness by achieving a score of 91.9% when the mislabeling ratio increases from 5% to 10%. On the TB dataset, incorrectly labelled logs similarly degrade the performance of the different methods for anomaly detection. Nevertheless, regardless of whether the mislabeling percentage is 5% or 10%, ilLog consistently exhibits superior precision, recall, and F1.

Figure 5 (e)-(f) presents the detection results of models utilizing different log parsers on the BGL and TB datasets. The performance of the models is influenced by the noise introduced by log parsing errors. Specifically, on the BGL dataset, when parsed by Drain, the F1 for LogRobust, LightLog, and ilLog is 81.5%, 85.3%, and 87.7%, respectively. However, when the dataset is parsed by Spell, the detection performance improves by 2.1%, 1.7%, and 0.2%, respectively. Notably, the proposed ilLog method consistently outperforms the other methods on both log parsers. For instance, on the TB dataset parsed by Drain, ilLog outperforms LogRobust and LightLog by 8.8% and 0.9%, respectively, in terms of the overall evaluation metric F1. On the TB dataset parsed by Spell, ilLog still leads LogRobust and LightLog by 2.9% and 1.5%, respectively.

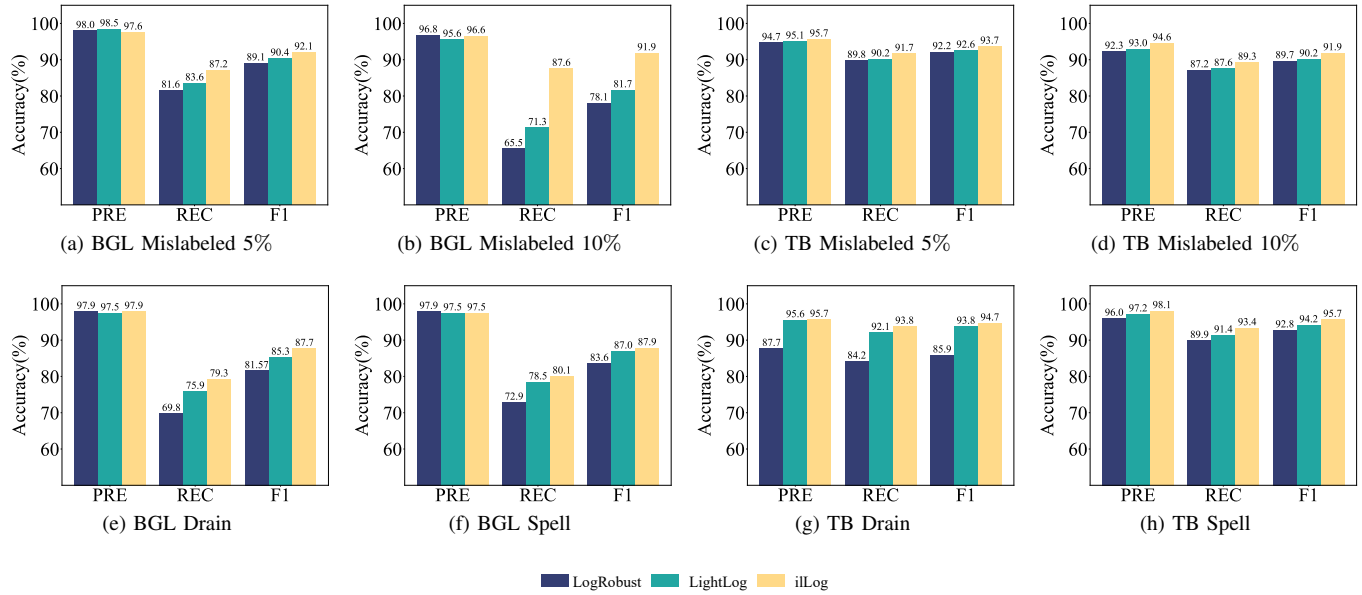


Fig. 5: Comparison of LAD methods with mislabeled and parsing noise

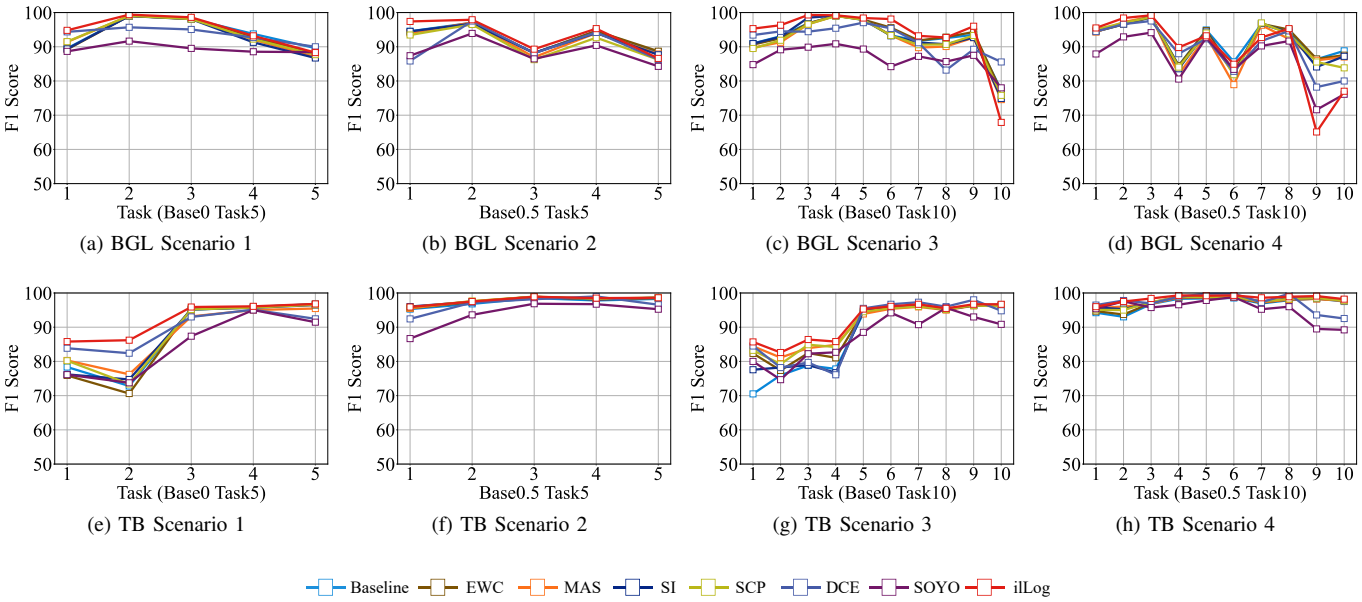


Fig. 6: Comparison of different incremental learning algorithms.

D. RQ3: Comparison with Different SOTA Incremental Learning Algorithms

To validate the advancement of the algorithm for incremental detection, experiments were conducted to compare the continuous detection performance of 8 incremental learning algorithms on 8 scenarios from 2 datasets. The Baseline algorithm adopts the same neural network model as ilLog but does not employ any incremental learning strategy. The algorithms compared fall into two main categories: one is task-incremental learning algorithms based on parameter regularization, such as EWC, MAS, SI, and SCP, which provide inspiration for addressing domain-incremental learning problems where the data distribution shifts while labels remain unchanged; the other is SOTA domain-incremental learning methods based on parameter isolation, i.e., DCE and SOYO.

As shown in Figure 6, in the field of log anomaly detection, the parameter isolation-based algorithm lags behind the parameter regularization-based algorithms. It is evident that in Figure 6 (a) (b) (c) (e) (f) (g) (h), ilLog exhibits a significant advantage in terms of overall detection performance. In Figure 6 (d), ilLog has lower detection performance on T9 and T10 compared to the Baseline and other 6 algorithms, but it still maintains a significant advantage in detection performance on T1-T8. Moreover, as an advanced algorithm in the field of

parameter isolation-based incremental learning, SOYO, which is based on model ensemble compression, achieves the lowest detection performance. Notably, the DCE algorithm, also based on parameter isolation, outperforms multiple parameter regularization-based algorithms in numerous scenarios, yet it still falls short compared to the proposed ilLog.

E. RQ4: Comparison Detection Performance with Different Backbones

TABLE III: Backbones Comparison

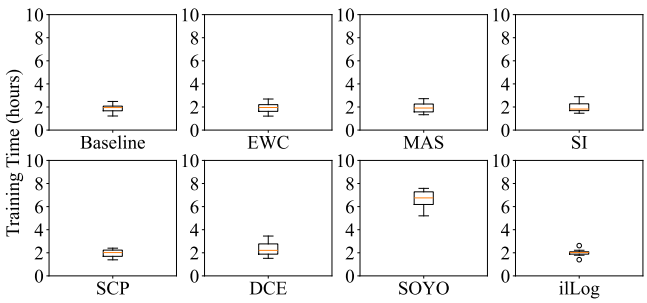
Backbones	B0T5		B0.5T5		B0T10		B0.5T10	
	BGL	TB	BGL	TB	BGL	TB	BGL	TB
LSTM	95.7	93.8	94.2	97.8	92.7	92.5	93.8	97.9
ResNet	91.4	88.8	89.9	95.3	93.4	88.7	90.1	96.1
Transformer	93.9	90.2	92.2	96.9	93.1	91.0	93.2	97.5
TSLANet	95.5	93.3	94.3	97.4	93.3	93.0	94.1	98.6
MLP	94.8	94.8	93.3	97.9	93.6	91.8	89.1	98.5

To evaluate the adaptability of the ilLog method across different neural network architectures, we select five backbones for analysis: LSTM, ResNet, Transformer, TSLAnet, and MLP. Specifically, LSTM is a classic choice for log anomaly detection due to its ability to model temporal dependencies in log sequences [6]–[8]; ResNet is commonly used as a backbone in numerous incremental learning methods [19], [38]; Transformer, as a mainstream architecture for sequence modeling, is widely adopted in log anomaly detection [18], [35] and incremental learning approaches [42]; TSLANet [48] is an advanced architecture specifically designed for temporal representation; additionally, a simple MLP model [43] is included as a baseline for comparison.

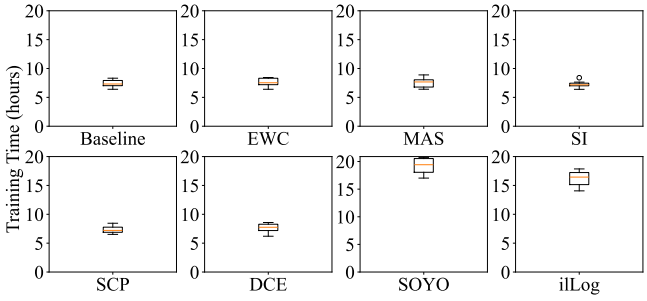
As shown in Table III, the performance of these backbones varies across different datasets and task scenarios. Overall, TSLANet demonstrates excellent and stable performance under most settings. LSTM achieves the second-best performance, attaining the highest F1 in the B0T5 scenario on the BGL dataset, which validates its effectiveness in modeling temporal relationships in logs. MLP performs outstandingly across all scenarios, highlighting the generalization capability of ilLog when paired with different backbones. In contrast, the performance of ResNet and Transformer shows slight fluctuations in specific tasks but remains at a high level in most cases. Experimental results indicate that the choice of backbone significantly impacts the final detection effectiveness of ilLog, with TSLANet and LSTM demonstrating stronger adaptability in incremental log anomaly detection tasks due to their architectural characteristics.

F. RQ5: Comparison Incremental Training Efficiency with Different SOTA Incremental Learning Algorithms

Given that incremental learning-based log anomaly detection methods necessitate continuous model updates, the computational complexity and time required for training become crucial considerations, particularly when facing vast training data and high-dimensional semantic vectors. Consequently, training time is an evaluation metric to be considered, and



(a) Training on BGL Dataset



(b) Training on TB Dataset

Fig. 7: Training time of different incremental learning methods in B0T10 scenario.

excessively complex incremental learning training methods may prove challenging for continuous online model updating. Therefore, to answer RQ5, we evaluate the training complexity of ilLog and 7 incremental learning algorithms on the BGL and TB datasets.

Taking the B0T10 incremental training scenario as an example, each subtask in the BGL dataset contained 5,708 training samples, while each subtask in the TB dataset contained 34,696 training samples. The dimension of the semantic vectors is 300, and the memory replayed memories were set to 10% of the training samples from the previous subtask. Each method underwent 10 training iterations. On the BGL dataset, the training time of the ilLog is similar to that of the baseline and classical parameter regularization-based methods. On the TB dataset, ilLog requires approximately twice the training time of the baseline but remains lower than that of the SOYO method, which is acceptable in practice. The results indicate that while effectively mitigating catastrophic forgetting and achieving advanced detection performance, ilLog does not introduce substantial additional computational overhead. Its training efficiency is comparable to that of a simple retraining strategy, demonstrating the practicality and advantage of ilLog in computational complexity and its suitability for meeting the demands of continuous model updates in online systems.

G. RQ6: Ablation Study

In this section, we conducted various detailed analyses of different components of ilLog to demonstrate their impact on classification performance.

TABLE IV: Results of ablation experiments with ilLog

Components				Test	B0T5	B0.5T5	B0T10	B0.5T10
SCP	QMC-SCP	ESR	Random Replay					
✓	✗	✗	✗	BGL	93.7	91.0	91.8	91.0
				TB	88.2	97.8	90.5	97.6
✓	✗	✗	✓	BGL	94.0	92.3	93.1	89.4
				TB	91.6	98.1	91.6	97.8
✓	✗	✓	✗	BGL	94.7	93.1	93.1	89.1
				TB	92.0	97.7	92.3	98.2
✗	✓	✗	✓	BGL	94.7	90.1	93.5	89.8
				TB	92.2	97.6	91.3	98.2
✗	✓	✓	✗	BGL	**94.8**	**93.3**	**93.6**	**89.1**
				TB	**92.2**	**97.9**	**91.8**	**98.4**

Table IV presents the influence of two main components, QMC-SCP and ESR algorithm. It can be observed that relying solely on SCP-based regularization loss to counteract forgetting and the no-compromise dilemma is insufficient for effective LAD. Given the settings as described in Section VI-A3, under the condition of an equal number of sampling points Q, the performance improvement of QMC-SCP is significant. The SCP-based method achieved the best performance of 91.0% in the BGL B0.5T5 testing scenario. However, it significantly lagged behind the ilLog detection method based on QMC-SCP and ESR in the remaining seven test scenarios. When SCP and ESR, as well as Random Replay, are combined separately, further performance improvements are achieved. The SCP based on ESR exhibited the best performance in the TB B0T10 test scenario, outperforming the SCP based on Random Replay in all test scenarios except TB B0.5T5. However, it lagged behind ilLog in the other seven test scenarios. The combination of QMC-SCP and Random Replay demonstrated superior performance compared to the original SCP. However, except for the BGL B0.5T10 test scenario, where it achieved 89.8%, surpassing ilLog's 89.2%, it still fell short in the remaining test scenarios compared to ilLog.

In summary, ilLog based on QMC-SCP and ESR achieved the best performance in the BGL B0T5, BGL B0.5T5, BGL B0T10, TB B0T5, and TB B0.5T10 test scenarios. In the TB B0.5T5 and TB B0T10 detection scenarios, ilLog exhibited detection performance of 97.9% and 91.8%, respectively. Although slightly lower than SCP+Random Replay (98.1%) and SCP+ESR (92.3%), it outperformed the other three combinations. Therefore, the proposed algorithm demonstrated the strongest detection advantage in terms of overall detection performance.

VII. Discussion

A. Why does ilLog work?

There are two main reasons why ilLog performs better than existing SOTA methods. First, ilLog enhances the effectiveness of the training set for log time sequences with discrete features through the implementation of the ESR algorithm, which can select training-value log samples from previous tasks. Second, ilLog incorporates a Quasi-Monte Carlo (QMC) algorithm based on Halton-based low-discrepancy sequences. This approach provides a more approximate value of the sliced Cramér distance while optimizing computational resource utilization.

Although ilLog achieves significant detection performance, it still has limitations. Graph neural networks have demonstrated strong modeling capabilities in log anomaly detection, effectively capturing complex interactions in distributed systems through graph structures and overcoming the limitations of traditional sequence detection models. However, the current implementation of ilLog mainly targets log event sequences, and its sample replay mechanism is more suitable for incremental learning of discrete sequences. In the future, we will explore how to extend entropy-driven replay to graph-structured data and validate the effectiveness of incremental learning on log graph, in order to handle complex scenarios where both log events and trace structures evolve.

B. Threats To Validity.

We have identified the following two major threats to validity.

Data Quality: The comparative experiments for the detection models were conducted using the BGL and Thunderbird public log datasets. Although these datasets have been extensively utilized in existing research, it is crucial to acknowledge the potential limitations of their data quality. These datasets were primarily collected and introduced in 2007, which implies that they might not encompass all the characteristics and patterns of the current system log evolution. Consequently, the proposed method may be limited in practical application.

Tool Comparison: We utilize the unified framework [27] based on PyTorch with default parameters for implementing DeepLog, LogAnomaly, LogRobust, and PLELog methods. For LightLog, since its original implementation was based on Keras, we converted it into a PyTorch-based implementation. For EWC, MAS, SI, SCP, DCE and SOYO methods, we reproduce based on the original paper and confirm that our results are similar to the reported values. However, there is a possibility of errors or inconsistencies in the conversion process, which could impact the fairness and accuracy of the tool comparison.

VIII. Conclusion

In this work, we introduced the problem of incremental learning for log anomaly detection. Specifically, we design a novel data replay algorithm that uses the entropy value of events to sort log sequence samples with discrete features and

selects samples with training values for subsequent training tasks, thereby reducing storage and computation requirements. Additionally, we improved a loss regularization based on the Cramér distance to calculate the distance between different probability distributions, mitigating catastrophic forgetting of knowledge. Our proposed method, ilLog, has been thoroughly evaluated through extensive experiments on three publicly available log datasets. Experimental results demonstrate the substantial advantages of our approach in handling continuously evolving log events within software systems.

REFERENCES

[1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, jul 2021.

[2] X. Wang, X. Zhang, L. Li, S. He, H. Zhang, Y. Liu, L. Zheng, Y. Kang, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "Spine: A scalable log parser with feedback guidance," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE'22, 2022, p. 1198–1208.

[3] N. Zhao, H. Wang, Z. Li, X. Peng, G. Wang, Z. Pan, Y. Wu, Z. Feng, X. Wen, W. Zhang *et al.*, "An empirical investigation of practical log anomaly detection for online service systems," in *Proceedings of the 29th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, ser. ESEC/FSE'21, 2021, pp. 1404–1415.

[4] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[5] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.

[6] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1285–1298.

[7] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, ser. IJCAI'19. AAAI Press, 2019, p. 4739–4745.

[8] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, J. Chen, X. He, R. Yao, J.-G. Lou, M. Chintalapati, F. Shen, and D. Zhang, "Robust log-based anomaly detection on unstable log data," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019, 2019, p. 807–817.

[9] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, and W. Zhang, "Semi-supervised log-based anomaly detection via probabilistic label estimation," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1448–1460.

[10] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "Metalog: Generalizable cross-system anomaly detection from logs with meta-learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024.

[11] X. Chai, H. Zhang, J. Zhang, Y. Sun, and S. K. Das, "Log sequence anomaly detection based on template and parameter parsing via bert," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 2, pp. 1150–1167, 2025.

[12] L. Chen, C. Song, X. Wang, D. Fu, and W. Zhou, "Csclog: A component subsequence correlation-aware log anomaly detection method," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6441–6453, 2025.

[13] Y. Huo, C. Lee, Y. Su, S. Shan, J. Liu, and M. R. Lyu, "Evlog: Identifying anomalous logs over software evolution," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 2023, pp. 391–402.

[14] X. Wang, J. Song, X. Zhang, J. Tang, W. Gao, and Q. Lin, "Logonline: A semi-supervised log-based anomaly detector aided with online learning mechanism," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 141–152.

[15] J. Tian, M. Li, Z. Wang, L. Chen, J. Qin, and R. Zhang, "Omlog: Online log anomaly detection for evolving system with meta-learning," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 30 142–30 155, 2025.

[16] J. Tian, M. Li, L. Chen, Z. Wang, X. Nie, and J. Qin, "Ssdalog: Semi-supervised domain adaptation for incremental log-based anomaly detection," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 6607–6619, 2025.

[17] M. D. Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, pp. 3366–3385, 2019.

[18] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "Swisslog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 2762–2780, 2023.

[19] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "icarl: Incremental classifier and representation learning," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5533–5542.

[20] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer International Publishing, 2018, pp. 556–572.

[21] B. Zhao, X. Xiao, G. Gan, B. Zhang, and S.-T. Xia, "Maintaining discrimination and fairness in class incremental learning," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 13 205–13 214.

[22] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[23] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer International Publishing, 2018, pp. 144–161.

[24] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, p. 3987–3995.

[25] S. Kolouri, N. A. Ketz, A. Soltoggio, and P. K. Pilly, "Sliced cramer synaptic consolidation for preserving deeply learned representations," in *International Conference on Learning Representations*, 2020.

[26] F. Wang, D. Zhou, H. Ye, and D. Zhan, "Foster: Feature boosting and compression for class-incremental learning," in *Computer Vision – ECCV 2022*. Cham: Springer Nature Switzerland, 2022, pp. 398–414.

[27] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proceedings of the 44th International Conference on Software Engineering*, 2022, p. 1356–1367.

[28] H. Studiawan, F. Sohel, and C. Payne, "Anomaly detection in operating system logs with deep learning-based sentiment analysis," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2136–2148, 2021.

[29] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Zhang, "Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 623–634.

[30] J. Li, H. He, S. Chen, D. Jin, and J. Yang, "Loggraph: Log event graph learning aided robust fine-grained anomaly diagnosis," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 1876–1889, 2024.

[31] J. Tian, M. Li, and J. Gan, "Fslog: Adversarial margin for cross-system few-shot log anomaly detection," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–16, 2026.

[32] J. Zhou, Y. Qian, Q. Zou, P. Liu, and J. Xiang, "Deepsyslog: Deep anomaly detection on syslog using sentence embedding and metadata," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3051–3061, 2022.

[33] B. Li, S. Ma, R. Deng, K.-K. R. Choo, and J. Yang, "Federated anomaly detection on system logs for the internet of things: A customizable and

communication-efficient approach," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1705–1716, 2022.
[34] Z. Wang, J. Tian, H. Fang, L. Chen, and J. Qin, "Lightlog: A lightweight temporal convolutional network for log anomaly detection on the edge," *Computer Networks*, vol. 203, p. 108616, 2022.
[35] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, 2022, p. 492–504.
[36] R. Chen, S. Zhang, D. Li, Y. Zhang, F. Guo, W. Meng, D. Pei, Y. Zhang, X. Chen, and Y. Liu, "Logtransfer: Cross-system log anomaly detection for software systems with transfer learning," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, 2020, pp. 37–47.
[37] G. M. van de Ven, T. Tuytelaars, and A. S. Tolias, "Three types of incremental learning," *Nature Machine Intelligence*, vol. 4, p. 1185–1197, 2022.
[38] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 12, pp. 2935–2947, 2017.
[39] Y. Wu, Y. Chen, L. Wang, Y. Ye, Z. Liu, Y. Guo, and Y. Fu, "Large scale incremental learning," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 374–382.
[40] K. Zhu, W. Zhai, Y. Cao, J. Luo, and Z.-J. Zha, "Self-sustaining representation expansion for non-exemplar class-incremental learning," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 9286–9295.
[41] S. Yan, J. Xie, and X. He, "Der: Dynamically expandable representation for class incremental learning," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 3013–3022.
[42] L. Li, D.-W. Zhou, H.-J. Ye, and D.-C. Zhan, "Addressing imbalanced domain-incremental learning through dual-balance collaborative experts," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: https://openreview.net/forum?id=dwjwvTwV3V
[43] Q. Wang, X. Song, Y. He, J. Han, C. Ding, X. Gao, and Y. Gong, "Boosting domain incremental learning: Selecting the optimal parameters is all you need," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 4839–4849.
[44] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*, 2016, pp. 859–864.
[45] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, ser. SOSP '09, 2009, p. 117–132.
[46] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, 2007, pp. 575–584.
[47] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.
[48] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "TSLANet: Rethinking transformers for time series representation learning," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 21–27 Jul 2024, pp. 12 409–12 428.

Jiyu Tian received the master's degree in cyberspace security from Dalian University, Dalian, China in 2022. He is currently pursuing the Ph.D. degree in Dalian University of Technology, Dalian, China. His current research focuses on temporal anomaly detection and cyberspace security.

Mingchu Li received his doctorate in Mathematics, University of Toronto in 1998. He worked for School of Software of Tianjin University as a full professor (from 2002 to 2004), for School of Software Technology of Dalian University of Technology as a full Professor and Vice Dean (from 2002 to 2023), for Jiangxi Normal University as a full Professor from 2024 to now. His main research interests include theoretical computer science and information security, and trust models and cooperative game theory.

Zumin Wang is a professor at Dalian University since 2014. His research interests include Sensors and Sensor Application, Wireless Sensor Networks, and Smart City. He received his MS degree in Mechanical Manufacturing and Automation in 2004 from North University of China, and Ph.D. degree in Physical Electronics in 2007 from the Chinese Academy of Sciences. He was a visiting scholar at the University of Washington from Nov. 2016 to Nov. 2017. He is a distinguished member of CCF.

Liming Chen received his B.Eng and M.Eng degrees from Beijing Institute of Technology, China, in 1985 and 1988 respectively, and his Ph.D. degree from De Montfort University, UK, in 2003. He is currently a Chair Professor at the School of Computer Science and Technology, Dalian University of Technology, China. His research interests include pervasive computing, data analytics, artificial intelligence, user-centred intelligent cyber-physical systems and their applications in smart healthcare and cyber security. He has over 300 publications in the aforementioned areas. Liming is an IET Fellow and a Senior Member of IEEE.

Jing Qin received the Ph.D degree in the School of Computer Science and Technology, Dalian University of Technology, Dalian, China. She is an associate professor in the School of Software Engineering, Dalian University. Her research interests include multimedia information retrieval, signal processing and machine learning.

Jianyuan Gan received the B.S. degree and the M.S. degree in science from Nanchang Hangkong University, Nanchang, China, in 2016 and 2019, respectively. He is currently pursuing the Eng.D. degree at Dalian University of Technology. His research interests include traffic engineering, AI.